

Individual Study I : Reinforcement Learning in Gaming

(Adaptive Boss fight)

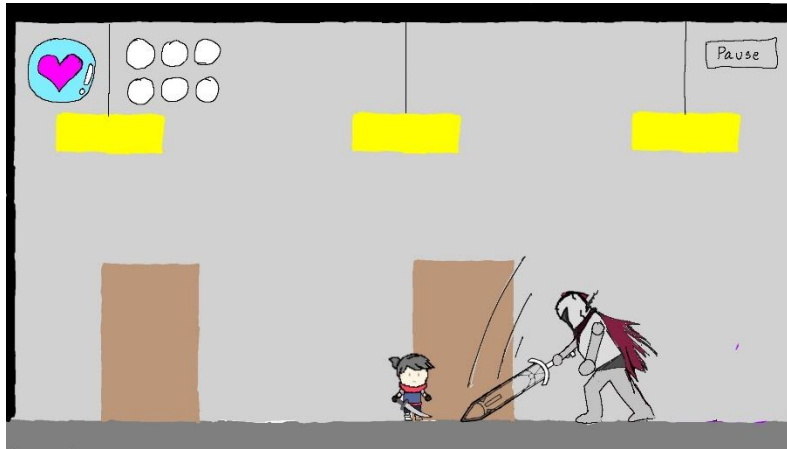
Objective



(Figure 1 : Footage from Hollow knight (Inspiration))

From my gaming experience, in an action game where there are challenging boss fights, many of them consist of movesets designed to catch unexpected players in various environments. Some players may have a different moveset that they are struggling with, including me. Sometimes, my friends and I have encountered a boss that keeps doing that moveset we struggled with. And then, once figured out, the enemy proceeded to use other movesets that troubled us once again. We have always joked about them being “adaptive AI” despite them not really being one. Therefore, I aim to make a boss fight that then uses an AI to learn players’ actions and try to adapt itself to use an effective moveset with effective attributes to always catch players off guard.

Introduction : Game Design



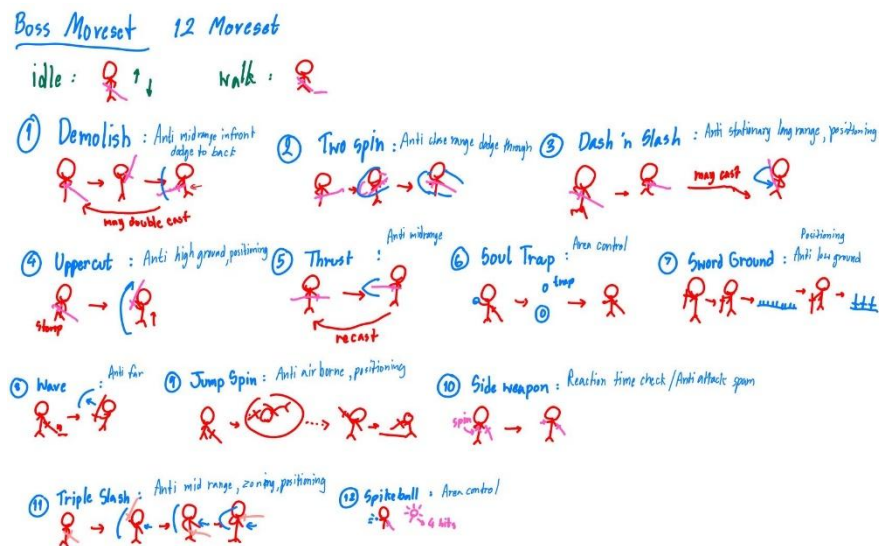
(Figure 2 : Game design Prototype)

The game will be an action platformer boss-rush style. It's going to be about a player fighting the same boss repeatedly, as both of them will get stronger each time. The boss will try to adapt to the player's playstyle by changing their moveset sequence, angle and timing of attack, and when or where to deploy traps to limit the player's movement. On the other hand, the player will receive buffs to gain more power, and when combined, they can allow the player to perform a special move to combat against adaptive boss fights.

This game is going to be an experimental type of game that asks the player to experience the adaptiveness of the boss. However, it may cause the player to be bored out of the game before the result has even taken place.

To counter this, this game will have a very smooth movement which gives a satisfying feeling when maneuvering across the area and gives freedom of control for the players. Furthermore, the game will provide a substantially powerful buffs after each win which will give even more freedom of movement and more window of failure to the players. However, it will not give the significant ability to shorten the fight such as attack damage or without a drawback to discourage the player as shorten the fight will result in shorten the training time for our AI.

The player character will have an ability to jump, double jump, attack, dodge, and dive down and can propel upward by striking down. The player can also grab off a ledge and may jump off ledge to ledge, which opens up a lot of freedom. The player will only have 1 life and a rechargeable shield protecting it. The player will have to hit the boss a set number of times (20) to defeat it. There is no way to reduce this number other than the additional ability obtained by acquiring the same buffs 5 times.



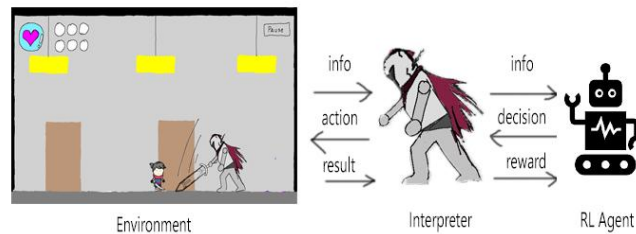
(Figure 3 : Boss movesets design)

The boss will have 12 movesets to choose from and can combo them in the same sequence of attack up to 4 times. Each moveset is designed to specialize in different situations and can be parameterized to change some aspects of the attack (angle, timing).

The game will require the player to defeat the same boss 10 times to win the game. Each time, the boss will actually adapt to counter the player's playstyle by external AI in Python. The boss will send out information about the environment to AI then the AI will give back the decision to the boss. After that, whether the boss hits the player or not, it will send back the corresponding reward back to AI. When the batch is ready, the AI is trained externally in Python during the fight, thus not interrupting the fight. Preferably, for consistency of the fight, the boss will use the old model during the fight and will change to a trained one in the next fight.

Chapter I: Studying RL

Once I have created a base game prototype that are enough for trying the AI in **Unity Engine**, I have to research information about reinforcement learning which I have a little prior knowledge beforehand.



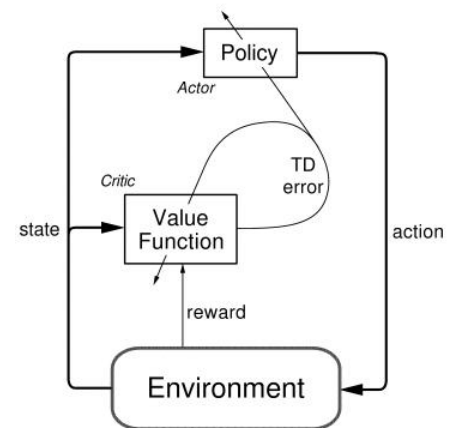
(Figure 4 : Boss RL workflow)

Reinforcement Learning (RL) is a type of machine learning that focuses on how agents can learn to make decisions through trial and error to maximize cumulative rewards. RL allows machines to learn by interacting with an environment and receiving feedback based on their actions.

The algorithm that will be used in the case is **Proximal Policy Optimization (PPO)**, a policy gradient method that optimizes the objective function of a policy directly.

I found out that PPO require 4 main components (and more that use in typical NN)

1. **Actor/Policy Network** to be the decision-making model
2. **Value/Critic Network** to estimates the expected cumulative reward of that state to help with learning
3. **Advantage Estimation** to estimate how better or worse an action compared to average
4. **Clipped Loss Function** the key feature of PPO. It uses to prevent major number of loss while calculating loss.



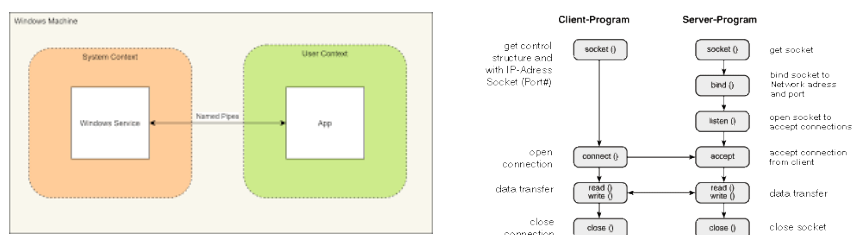
(Figure 5 : PPO workflow)

To implement my own PPO in Python, I try to learn about it from https://pytorch.org/tutorials/intermediate/reinforcement_q_learning.html. Though, without enough knowledge of neural networks in general, I was struggling to understand it; thus, it took me a long time to get what function does what. Moreover, all of the tutorial revolves around solving OpenAI Gym problems, which do not relate to my work at all, and I have to learn about environment implementation, which I didn't use.

However, I end up using others' codes for reference instead, which integrate raw GAE (Generalized Advantage Estimation) and clipped weight calculation without using PyTorch utility (torchrl). It ends up being better since I can implement my own environment pipeline into it without needing to ponder around with env transformation stuff.

Then, I have to implement the way for Unity and external Python to communicate.

- At first, I tried to find a built-in solution for Unity, but it didn't work out for me.
- I tried to research a method to communicate between two applications, and I found several ways, including TCP sockets and named pipes.
- I try using a named pipe because I have some doubt about TCP in-computer connection.
- Because of the inconsistency of named pipes, I have to go with the TCP socket instead, which is surprisingly very effective.



(Figure 6,7 : Application to Application connection)

At first, the input parameter was

- *player position (X, Y)*
- *boss position (X, Y)*
- *Player Velocity (X, Y)*
- *number of attacks in this combo*

- *an array for each moveset of the number of times it has been executed in the current combo*
- *the last moveset executed.*

And the output was

- *moveset number*
- *telegraph time*
- *attack time*
- *recovery time*
- *angle*
- *distance*
- *trap X*
- *trap Y.*

Later, I found out that there were too many, so I removed many unnecessary parameters until I was left with

Input : PlayerPos, BossPos, PlayerVc

Output: moveset number, attack time, angle (fewer parameters for the boss to adapt), and trap position (which I later removed and left as random).

The training will be an ensemble training in which the boss will project the invisible bosses for each attack that has no interaction with the player but can check if the attack hit or not. The boss may use different movesets or other variations to explore the environment. Therefore, the external Python must handle multiple agents, but they still share the same memory and train together simultaneously. But preferably, the main boss should use the old model.



(Figure 8 : Ensemble training in action)

The workflow works like this.

- Unity will host the socket and wait for an external Python script to connect.
- When the idle time has ended, the game allows the boss to attack.
- The boss and its projection will ask for a decision by sending information in the game through a TCP socket to an external PPO.
- When sending a message, the boss script will collect information in the game from a static singleton and then pack it into an array-like string. The string will include a header including commands for AI ([Command, Agent Number]).
- The Python script will extract the message, branch it to the correct agent, and send the input to AI to predict while saving the information in the buffer.
- After it finishes predicting the action, the Python script will save the action and then send it back to Unity as a string.
- After the Unity side receives the message, it sends the decision back to the corresponding boss to take action.
- The boss will take action and check the hit result. Whether it hits or hits while the player is dodging or does not hit at all.
- The boss will send the corresponding reward back to AI with a different command.
- The Python script then sends the message to the correct agent.
- The agent will complete its buffer and send the buffer to the shared memory and check if the branch is ready to be trained.
- If it is ready to train, then it will train asynchronously with the reserved model. When finished, the model is saved and can be loaded for other agents.

The environment to be trained started with player standing still and teleporting to random location upon getting hit. If the result is satisfying, the dummy player will be coded to run around the area. Then will try to dodge the attack. Finally, I will train the AI myself.



(Figure 9 : Boss attack miss)

The result was unsurprisingly unsuccessful. I have trained the model as much as I could when I open my laptop. It is estimated to have more than $50,000 * 20$ epoch of training. With the expected pattern is the boss using only dash or wave attack which can hit the player wherever both of them are. The boss still behaves like it was random but sometime does exceptionally well. The reason may be because of my lack of experience on this field since I didn't understand some of the functions. There are errors along the way that I fix by hard coded many times. And finally, the hyperparameter may not suited my project and since the training period is really long, manually adjusting it is very tiring and inefficient for me.

Chapter II: PPO struggle and mini environment test

Since my lack of some understanding of my model (if the neural network is corrected or not), with a doubt in mind, I propose a small test for my model. By creating a mini environment that has some similarity to the target environment, it is an AI that decides which action to take in a not-so-continuous environment.



(Figure 10 : Design of the mini environment)

A text-based game involved the player attacking the enemy while being aware of when the enemy would attack and then defending it according to the damage done. The player will win if the enemy's HP reaches zero but will lose if the player's HP reaches zero instead (if they both drain simultaneously, it counts as a win for the player). For detail, the player's HP will always start out as 5 HP. At the start, the enemy will randomize their HP, attack damage, and delay. When it is the player's turn, the player may choose between 4 actions.

- Attack allows the player to deal 1 hp damage to the enemy.
- Strong Attack allows the player to deal 3 hp damage instead but will skip a turn.
- Defend will reduce the incoming attack by 3
- Counter will be activated if the enemy attack the player with 3 or more damage, otherwise will do nothing. It will cancel the damage and strike back with 1 hp damage.

When it is the enemy's turn, it will reduce its delay by one. If the delay reaches zero, it will attack the player, dealing its attack damage as damage. Then, the enemy will randomize its attack damage and delay again.

The reward setup is the AI will gain/lose the reward for how many damages dealt or taken in order. If the player manages to defend against the enemy, the reward is 1—damages leak from defending (4 or more). And if the player manages to successfully counter the enemy, it will give 2 rewards. If the player defends or counters without the enemy attacking them, it gives -1 reward. Additionally, if the player wins the fight, it will give 5 rewards. Otherwise, it will give -5 rewards if the player loses.

Without much thinking, the correct strategy would be attacking the enemy until its delay reaches one, then performing a defensive option accordingly. For more damage output, the strong attack should be used when the enemy's delay is at 3 or more or when the enemy is lethal to a strong attack.

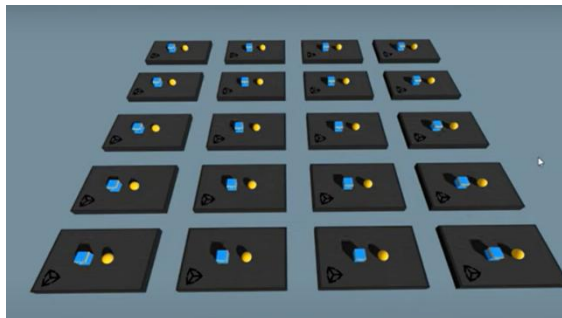
The first environment test setup is the enemy will have a range of HP of [1, 20], attack damage of [1, 5], and a delay of [1, 3]. A very standard enemy to train. After the training, here is the result. After 10,000 fights, it seems to form a solid strategy of “Keep blocking and counter until the enemy dies.” Which is a very safe method, but it was flawed on my part to give too generous a reward for defending (even taking damage during defending still doesn't lose that much reward).

And after 50,000 fights, the average reward gain was 8 rewards per fight and a max win rate of 80%. However, the average reward gain was not a good evaluation method since it can fluctuate by random enemy's HP. For enemy that has 1 HP at the start, the maximum reward would be 6-8. On the other hands, the enemy has full 15 HP, the maximum reward will spike up to a minimum of 20.

After a rather pleasing outcome, I change the environment setup to see how adaptive the model can be. By changing the random range of the enemy to [5, 20] HP, [1, 4] attack damage, and [1, 2] delay. With this setup, the enemy will become more tanky, less powerful, but faster. It will shut down the option of strong attack since the delay is 2 or less now.

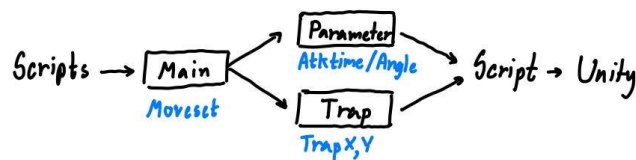
After 30,000 fights, the win rate and average reward become stable again, but with decreased performance of a 60% win rate and 7 rewards per fight (higher enemy HP). At this point, it begins to foreshadow to me that for AI to adapt to a new environment setup, it would take a very long time. However, I still keep going on this subject; thus, this report will get much longer.

At this point, I was suggested to use a better way to do ensemble training, which is to split environment. Although my environment was hard-coded for the boss to send out projections and player information was kept in a singleton instance, making the change complicated, I tried it anyway. However, the message sent was too much for TCP and too many threads for agents, causing several errors. I had to back off. I should have turned off the boss projection mechanic, but I thought it was required because I thought it was the only way to implement ensemble training while the player was playing in the game.



(Figure 11 : Multi Environment Ensemble Training)

Since the problem was clear, the output space is too large. So, I split the model into 3 models. The main moveset model, the attack parameter model, and the trap position model. They have separate models, optimizers, memory, and reward systems but are linked in some ways. The workflow will now work like this. After the script has received the message, it sends it to the main model first. The main model decides the moveset to use, then it sends the moveset with the message to the parameter model if it is not a trap moveset; otherwise, it sends it to the trap model. The parameter model will decide the attack time and the angle of the attack, and the trap model will decide some of the trap locations. Each time the player gets hit by the trap, it accumulates the trap reward and then send it once the trap is repositioned.



(Figure 12 : Multi Model design)

I believed this method would reduce the overhead of the AI since instead of outputting every parameter for every moveset, it will decide the parameter necessary for that moveset instead. Unfortunately, after this change, the AI still didn't show a significant change. Maybe it is because I didn't evaluate it properly, like making a loss graph, but rather sometimes peeked to see the performance in some timeframes. But I still think that if I peek many times and I haven't seen it get better, then I guess it probably won't.

Chapter III : Trying Different Method

At this point, I'm in disbelief that my model is capable of being a successful child. So, I have to come up with different methods of making the boss adaptive.

First, I start off by thinking method that inspired off a reinforcement learning or the method that reinforcement learning is inspired of. Then, I thought, what if I scale down to a state-based SoftMax solution? The idea is to have each state of the environment have its own SoftMax memory to decide. Since it didn't use the neural network to handle an endless amount of state. I only classify all the states into 27 states based on the positional difference between the player and the boss.

Position Different				
Left		Top		Close
Right	X	Bottom	X	Mid
Zero		Zero		Far

(Figure 13 : State Mapping)

And then I implement a savable memory for each state containing each logits for each moveset or parameter. The memory size is exceedingly large, making it very difficult to manually inspect the logits of each state. With my naivety, I let it train, not knowing if it's actually working or not.

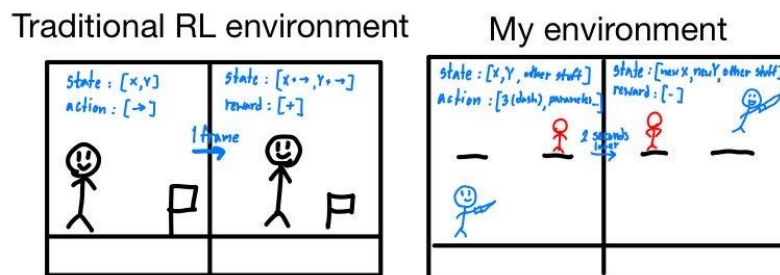
After some time had passed, I found out that it doesn't work at all, worse than my old PPO approach. So, I try to check it out by limiting the moveset to only dash to see if even the parameter SoftMax is doing good. At the end, it turns out that even though the memory and the SoftMax algorithm are working fine, the boss still doesn't improve. With time running out, I have no choice but to try and use the built-in package of pretrained models ready to be used.



(Figure 14 : Unity MLAgent Title Card)

The **Unity Machine Learning Agents Toolkit** (ML-Agents) is an open-source project that enables games and simulations to serve as environments for training intelligent agents provided with implementations (based on PyTorch) of state-of-the-art algorithms to enable game developers and hobbyists to easily train intelligent agents for 2D, 3D and VR/AR games. Provided with simple-to-use Python API to train Agents using reinforcement learning, imitation learning, neuroevolution, or any other methods.

This project is extremely convenient to use the AI for gaming in Unity since it is a package in Unity, which makes it easy to implement; just input the state, use the decision, and send the reward back. A simple framework that is suitable for traditional reinforcement learning jobs, except for the fact that I was having a doubt if my work is even suitable for reinforcement learning to begin with. Because of the non-continuous nature of my environment, I have to improvise from the tutorial a lot, to the point where I am unsure if it will work or not.



(Figure 15 : Difference of environment)

The first major problem is that because of MLAgent was designed to train the AI in the development phase to use in the game, not train during play time. So, the default config is to speed the engine up by 20 times. That makes observing impossible, and it makes weird physics too. When I edit the config to an observable state, the physics seem to slow down, making some of the boss movesets unusable, such as the effective dash moveset. Although it can't be used like this in playtime without solving the problem, such as turning all physics into position-based math, I decided to test the AI anyway because of the time I have left with this project.

The result was noticeable but bizarre. The boss keeps using its vertical jump attack until it hits the player. It seems very ineffective, but at least its policy is still looking solid. I could point out some of the reasons it would work like this. One of them is the player position, which appears in a hard-to-reach location. About 10% of the randomized locations can lead to the player being on the top platform, which limits the options that the boss can do. But there are movesets that are much more effective than jumping vertically even though the player is beneath them.

The other reasons might be because the jumping moveset doesn't require a crucial angle parameter like the dashing or projectile one. Making it easier to reach the maxima even though it is a local maxima.

Conclusion : Failure, Reason, and Future Learning

With my inexperience about reinforcement learning or even neural network models before signing up for this study, I have failed in making adaptive AI that I was eager to try out. Still, it was an insightful experience knowing how things work and knowing why they don't work. I have learnt so much about reinforcement learning that I even use it as my technical presentation in my English class. Other than that, I have learnt about application-to-application communication, such as named pipes and TCP sockets, and a lot of Python and PyTorch stuff.

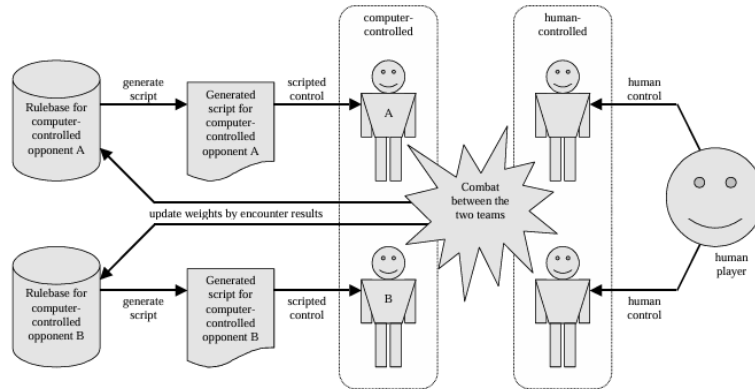
Later, before I closed this project, after I had studied about AI in the class, I tried to analyze my old PPO code. I have found some flaws and am trying to fix them. Then I tried to train it in my mini environment. Although, it does improve the performance slightly by achieving the maximum win rate of 90 % at 30,000 fights. Still, I don't think it is enough for my game environment because of the sole reason I have found.

It adapts too slowly to use on hyper-adaptive humans. Even in my mini environment, it takes over 10,000 tries to adapt to compete against the different set of enemies, making it seem unlikely that it will adapt to humans before humans win against them 10 or 20 times. Or it is because my model was even more flawed than I expected, but that is for my future learning.

Lastly, after I decided to give up on this project for now, I was suggested to use **Dynamic Scripting** for my project.

Dynamic scripting is a technique that realizes unsupervised online adaptation of opponent AI in complex commercial computer games such as CRPGs. It is based on the automatic online generation of AI scripts for computer game opponents by means of an adaptive rulebase.

(Ref:<https://www.spronck.net/pubs/EnhancingThePerformanceOfDynamicScriptingInComputerGames.pdf>)



(Figure 16 : Diagram of Dynamic Programming)

I'm pretty sure that dynamic programming will be the solution to my project, but it is not going to be a true adaptive AI in boss fights anymore. Sadly, I have run out of time since I have to focus on my CG project in my CG class, so if I have time or the next time I get interested in this project again, I won't hesitate to try dynamic programming out to improve the quality of the boss fight.

Lastly, the project was a wild ride for me. I have learned many things doing this study. I have struggled many times doing this project. However, during this study, I realized how much I miss creating a game design. So, my next study is unlikely to be a continuation of this project.

Finally, I want to thank my instructor for leading me to this point by suggesting many helpful solutions throughout this study. I may end up in failure, but I still learn a lot about reinforcement learning.

Github : <https://github.com/lolpopih845/RLinGamingTest>

Rangsiman Jearuksa 6633208521